Technical University of Crete
School of Electrical and Computer Engineering
Course: Optimization
Exercise 2 (130/1000)
Report Delivery Date: 20/11/2023
Instructor: Athanasios P. Liavas

Paraskevopoulou Despoina
2021030135

In this exercise, we will contemplate simple unconstrained convex optimization problems.

A. We consider a simple quadratic optimization problem.

(a) Let $f : \mathbb{R}^2 \to \mathbb{R}$. For fixed $\mathbf{x} \in \mathbb{R}^n$ and $m > 0$.

i. We proceed to compute the $\nabla g_x(\mathbf{y})$ as follows

$$\nabla g_x(\mathbf{y}) = \nabla(f(x) + \nabla f(x)^T(y - x) + \frac{m}{2} \parallel y - x \parallel_2^2) \Rightarrow$$

$$\nabla g_x(\mathbf{y}) = \nabla f(x)^T + m(y - x)$$

ii. The optimal point satisfies this statement $y_* = argmin_y g_x(\mathbf{y})$. This is the point where

$$\nabla g_x(\mathbf{y}) = 0 \Rightarrow$$

$$\nabla f(x) + my - mx = 0 \Rightarrow$$

$$my = mx - \nabla f(x) \Rightarrow$$

$$y_* = x - \frac{\nabla f(x)}{m}$$

So, the optimal value $g_x(\mathbf{y}_*)$:

$$g_x(\mathbf{y}_*) = f(x) + \nabla f(x)^T(x - \frac{\nabla f(x)}{m} - x) + \frac{m}{2} \parallel x - \frac{\nabla f(x)}{m} - x \parallel_2^2 \Rightarrow$$

$$g_x(\mathbf{y}_*) = f(x) + \nabla f(x)^T(-\frac{\nabla f(x)}{m}) + \frac{m}{2} \parallel -\frac{\nabla f(x)}{m} \parallel_2^2 \Rightarrow$$

$$g_x(\mathbf{y}_*) = f(x) - \frac{1}{m}\nabla^2 f(x) + \frac{1}{2m} \parallel \nabla f(x) \parallel_2^2$$

B: *Answer:*

For the first part of this exercise, we construct the random orthonormal $\mathbf{U}$, as it was indicated, by creating a random $(n \times n)$ matrix $\mathbf{A}$ and then we computed its singular value decomposition. Subsequently, we computed $U * U'$ and $U' * U$ and we observed that we derived the same values.

Code:

```
%% B
close all;
clear all;
n = 2;
A = randn(n,n);
[U,S,V] = svd(A);

% they are the same
one = U*U';
two = (U')*U;
```

Then, for question b, we constructed the eigenvalues λ_i for $i = 1, ..n$ and from those λ_{max} and λ_{min} were selected. With that being done we have the condition number of the problem as $K = \frac{\lambda_{max}}{\lambda_{min}}$.

Code:

```
lmin = 10;
lmax = 1000;
z = lmin + (lmax - lmin) * rand(n-2,1);
eigP = [lmin;lmax;z];
L = diag(eigP);

condN = lmax/lmin;
K = condN;

P = U*L*U';
%eig(P)
q = randn(n,1);
```

Following that, we proceeded to solve the problem, using closed-form solution as

$$\nabla f(x) = Px + q$$

$$x_{optimal} = argmin_x f(x)$$
$$\nabla f(x_{optimal}) = 0 \Rightarrow$$
$$x_{optimal} = -\frac{q}{P}$$

Then, we moved on to compute the optimal value $p_* = f(x_*)$.
Code:

```
% iii
gradf = @(x) P*x + q;
x_opt = -q'/P;

x_opt = x_opt'
fstar = (0.5)*(x_opt')*P*x_opt + (q')*x_opt
```

The result of fstar is constantly changing due to the use of the randn functions for P and q. Moving on, we solved the problem using cvx, and got the same solution as above.
Now we go on to solve the same problem using the gradient algorithm, with exact and bactracking line search. Below, are presented the figures and the values for a run with condition number $K = 1$. For that run the iterations necessary for convergence using the exact line were 2.

```
Status: Solved
Optimal value (cvx_optval): -0.0210601

Optimal value f = -0.021060
Iterations needed for exact line = 2
Optimal value f = -0.021060
Iterations needed for backtracking = 9
```

Figure 1: The solutions of cvx and the two algorithms

Code:

```
% v
% exact line search
k = 1;
e = 0.0001;
a = @(x) (gradf(x)')*P*(gradf(x));
b = @(x) norm(gradf(x))^2;
f = @(x) (0.5)*(x')*P*x + (q')*x;

x_k = [1;1];
all_x_k = zeros(2,k);
f_val = zeros(1, k);

while(norm(gradf(x_k)) > e)
    dx_k = -(gradf(x_k));
    t_star = b(x_k)/a(x_k);
    all_x_k(:,k) = x_k;
    f_val(k) = f(x_k);
    x_k = x_k + t_star*dx_k;
    k = k+1;
end
fprintf('Optimal value f = %f \n',f(x_k));
fprintf('Iterations needed for exact line = %d \n',k);

% backtracking
x0 = [1;1];
step = 1;
a = 0.2;
b = 0.5;
e = 0.0001;
[x,fun_val,f_val_b,all_x_kb,iter] =
gradient_method_backtracking(f,gradf,x0,step,a,b,e);

% backtracking function
function [x,fun_val,f_val,all_x_kb,iter]=
gradient_method_backtracking(f,g,x0,s,alpha,beta,epsilon)
x=x0;
grad=g(x);
```

```
fun_val=f(x);
iter=0;
all_x_kb = zeros(2,iter);
f_val = zeros(1, iter);
while (norm(grad)>epsilon)
    iter=iter+1;
    t=s;
    while (fun_val-f(x-t*grad) < (alpha*t*norm(grad)^2))
        t=beta*t;
    end
    all_x_kb(:,iter) = x;
    f_val(iter) = f(x);
    x=x-t*grad;
    fun_val=f(x);
    grad=g(x);
end
fprintf('Optimal value f = %f \n', fun_val);
fprintf('Iterations needed for backtracking = %d \n',iter);
end
```

The function that was implemented for the backtracking line search was used from the book "Amir Beck - Introduction to Nonlinear Optimization".
For n=2 and $K = 1$ we derived the following results using the contour function.

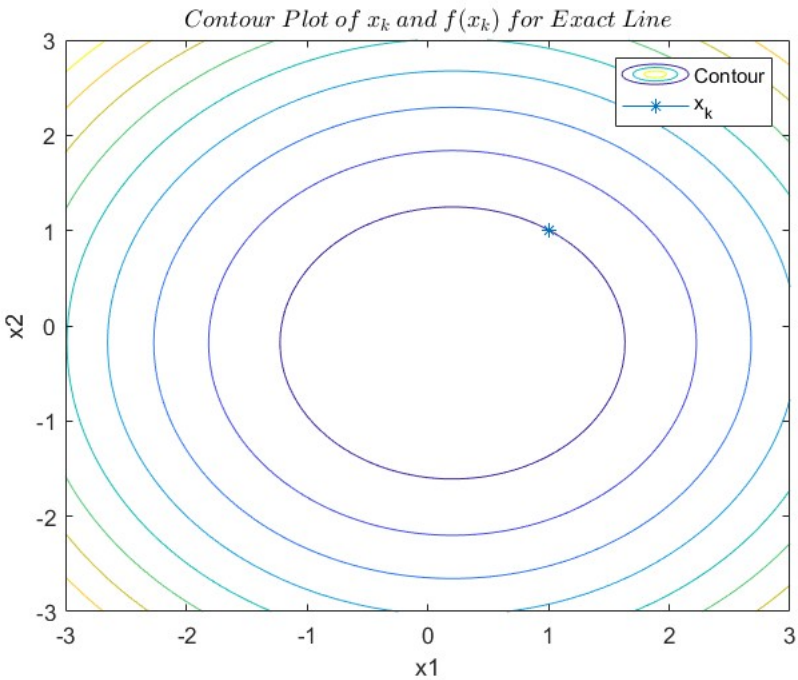

Figure 2: Trajectories of xk for exact line search (K=1)

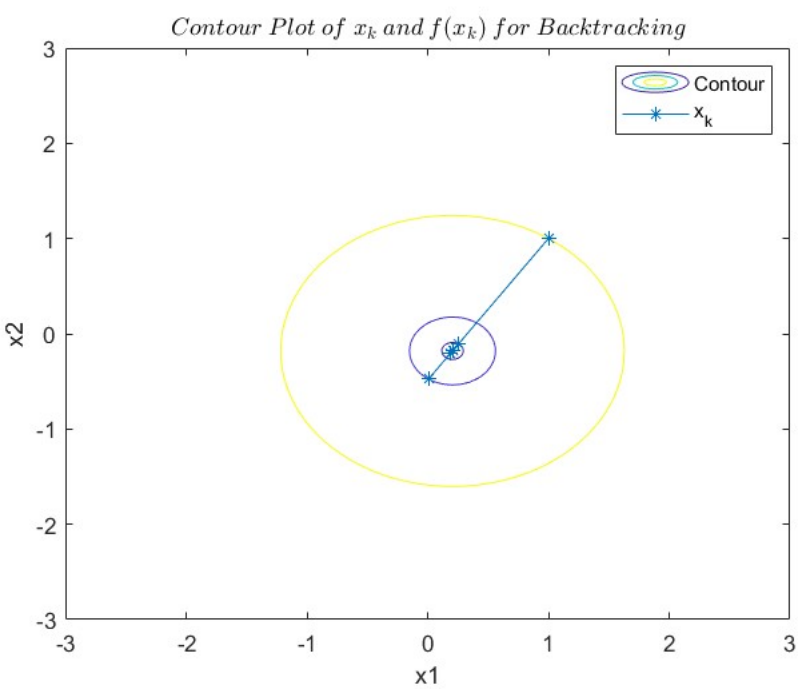

Figure 3: Trajectories of xk for backtracking line search (K=1)

Code:

```
% vi
x1 = -3:0.01:3;
x2 = -3:0.01:3;
%x_v = [x1; x2];
fc = zeros(size(x1,2),size(x2,2));
for i=1:size(x1,2)
    for j = 1:size(x2,2)
        fc(i,j) = (0.5)*([x1(i) x2(j)])*P*([x1(i) x2(j)]') + (q')*([x1(i) x2(j)]');
    end
end
contour(x1, x2, fc', f_val);
hold on;
plot(all_x_k(1, :), all_x_k(2, :), '*-');
xlabel('x1');
ylabel('x2');
title('Contour Plot of x_k and f(x_k) for exact line search');
legend('Contour', 'x_k');
hold off;

figure;
contour(x1,x2,fc' , f_val_b);
hold on;
plot(all_x_kb(1, :), all_x_kb(2, :), '*-');
xlabel('x1');
ylabel('x2');
title('Contour Plot of x_k and f(x_k) for Backtracking');
legend('Contour', 'x_k');
hold off;
```

In order to get a better feeling and a better understanding for the x_k and its values, we run the code for bigger K.

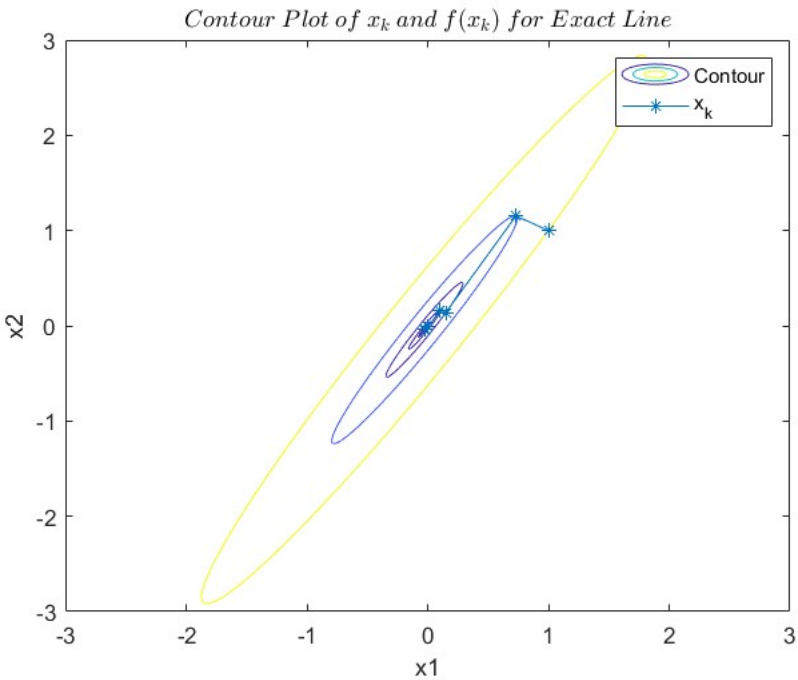

Figure 4: Trajectories of xk for exact line search (K=100)

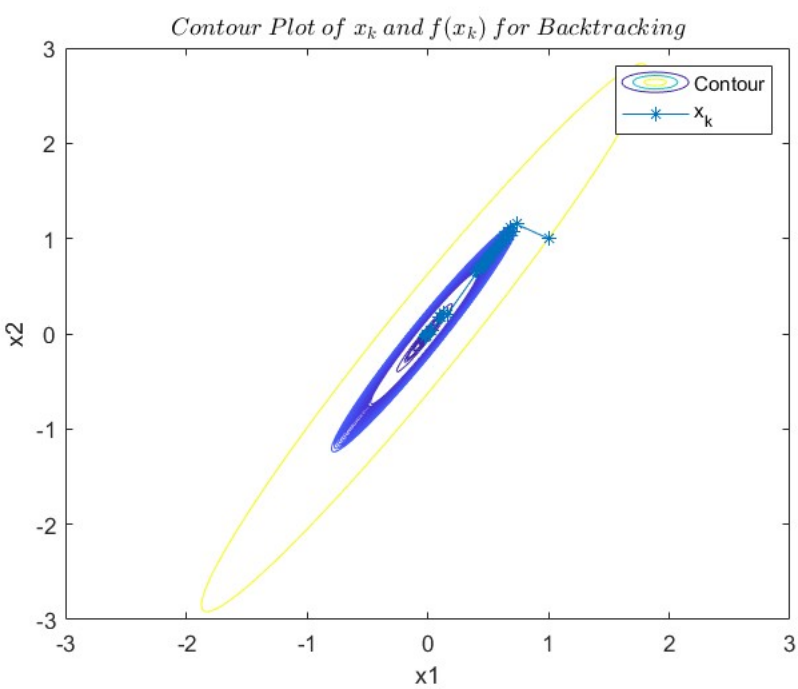

Figure 5: Trajectories of xk for backtracking line search (K=100)

If we observe closely these figures, we notice the "zig-zag" effect that happens when we connect the trajectories.

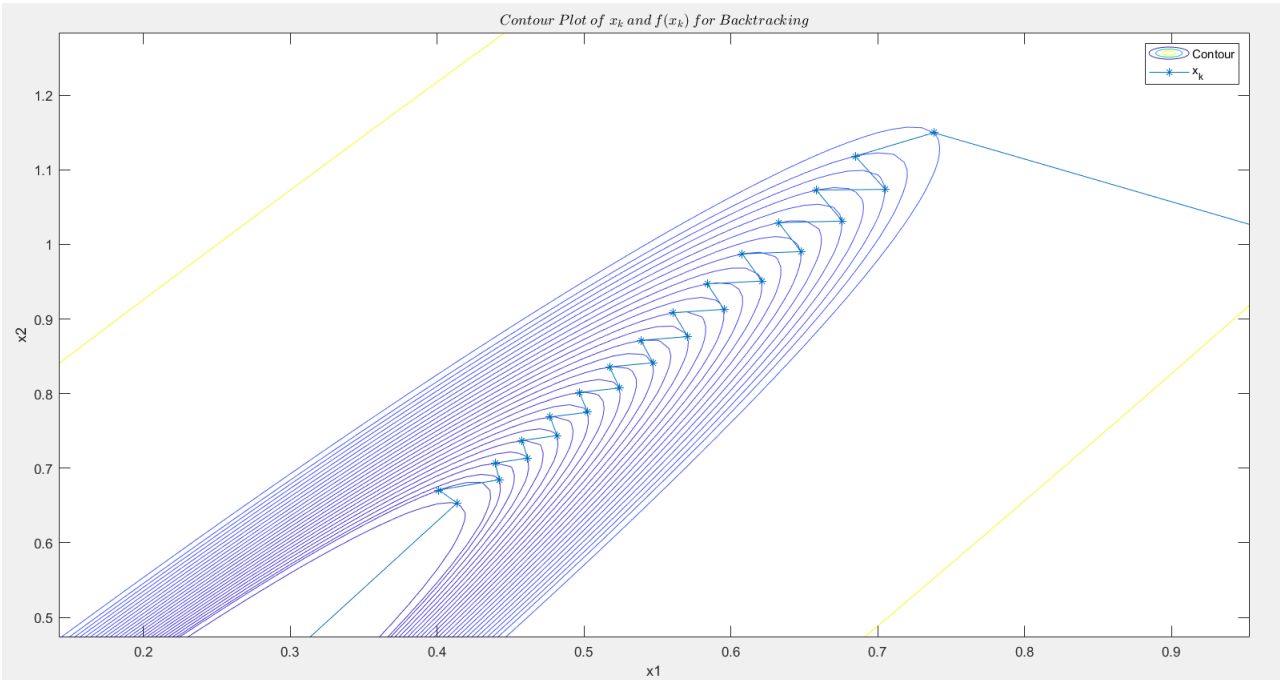

Figure 6: A closer look to the backtracking trajectories

Then, we moved on to the plotting of the quantinty $log(f(x_k) - p_*)$ produced by the two algorithms, vesrus k, the number of iterations.

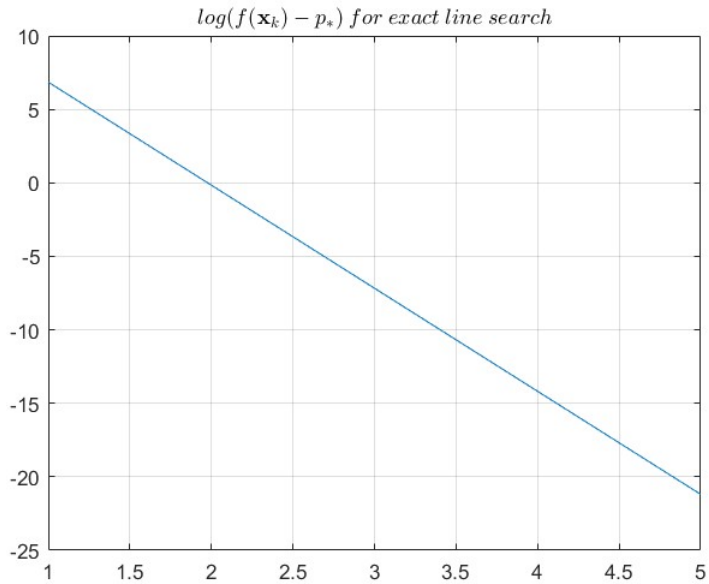

Figure 7: The quantinty $log(f(x_k) - p_*)$ versus the number of iterations for exact line search

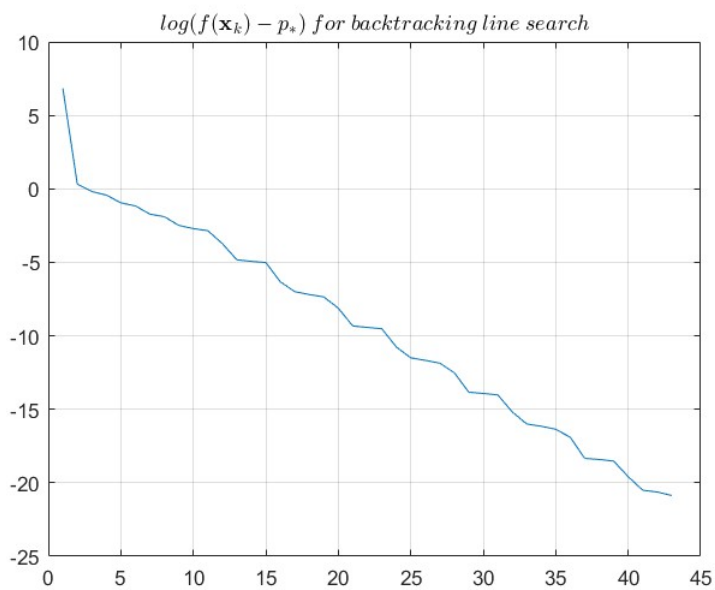

Figure 8: The quantinty $log(f(x_k) - p_*)$ versus the number of iterations for backtracking line search

The fact that for the exact line method the computed quantity versus the iterations is a line shows that there is a linear relationship between the quantity and the iterations. This is observed in the exact line because as an algorithm is precise, directly to the optimal point while the backtracking is an inexact technique. Also, we observe that the backtracking method begins with an intense drop.
For bigger values of the condition number we observe that the iterations for both algorithms increase. Though the iterations of the exact line are approximately 6 or 8, those of the backtracking is number of the order of thousands (3000 for example). By observing the plots, we see that for the exact line is the same straight line, but for the backtracking it appears that an almost straight line is also plotted. This implies that by increasing the condition number of the problem the two algorithms are slightly converging into the same. Also, another observation is that the backtracking search is slower, so it has higher time complexity.

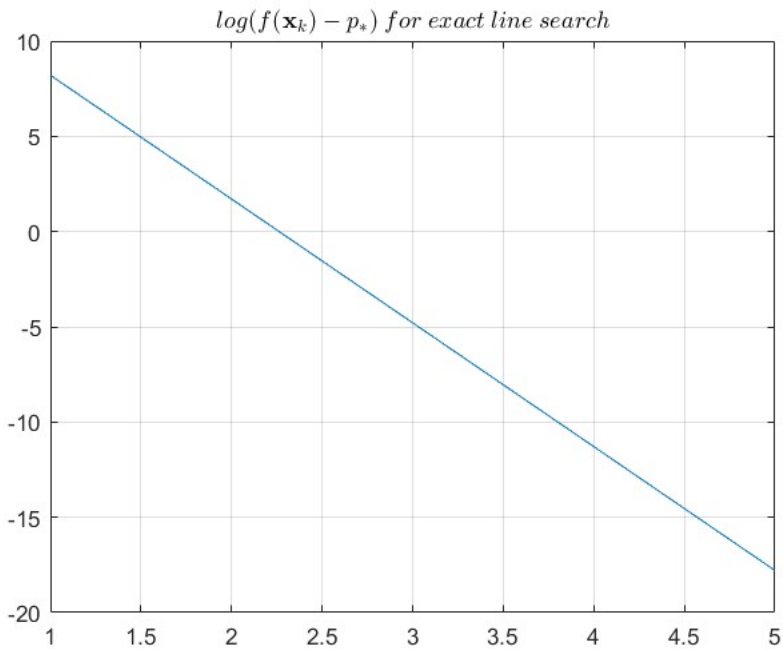

Figure 9: The quantinty $log(f(x_k) - p_*)$ versus the number of iterations for exact line search (K=1000)

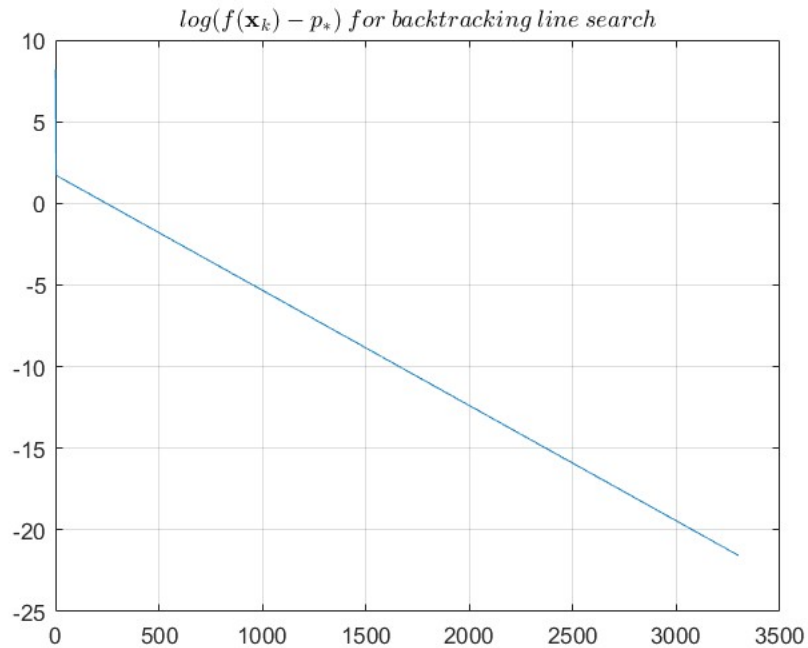

Figure 10: The quantinty $log(f(x_k) - p_*)$ versus the number of iterations for backtracking line search (K=1000)

The line for the backtracking isn't full straight, there are edges, but not so intense as they were for $K = 100$.
Code:

```
% vii
k_axis = 1:k-1;
iter_axis = 1:iter-1;
quant_ex_line = log(f_val(1:k-1) - fstar);
quant_back_line = log(f_val_b(1:iter-1) - fstar);

figure;
plot(k_axis,quant_ex_line);
grid on;
title('$log(f(\mathbf{x}_k) - p_*)\: for \:exact \:line\: search
$','Interpreter','latex');

figure;
plot(iter_axis,quant_back_line);
grid on;
```

```
title('$log(f(\mathbf{x}_k) - p_*)\: for \: backtracking \:line\: search
$','Interpreter','latex');
```

For the last question, we used the formula for the iterations that indicates that $k = \frac{log(\frac{f(x_0)-p_*}{e})}{log(\frac{1}{c})}$ where x_0 is the starting point, p_* is the value of f for the optimal x and $\frac{1}{c} = -log(1 - K^{-1})$. We wrote that in Matlab as

```
k_e= ceil(log((f(x0)-f(x_optimal))/e)/-log(1-(condN)^(-1)))
```

and the result we got, if $K = 10$ was i.e. 127, for $p_* = -0.091509$ and $e = 0.0001$.

C. Moving on to the third part of the exercise, we have the following function

$$f(\mathbf{x}) = \mathbf{c}^T\mathbf{x} - \sum_{i=1}^{m} log(b_i - \mathbf{a}_i^T\mathbf{x})$$

First, we are asked to prove that **domf** is convex. The proof is as follows:
We consider the points x_1, x_2 and their combination $\theta x_1 + (1-\theta)x_2$. We need to show that $\theta x_1 + (1-\theta)x_2$ (their convex combination) belongs to **domf**. So, we consider $b_i - a_i^T x_1$ and $b_i - a_i^T x_2$. We have that

$$b_i - a_i^T(\theta x_1 + (1-\theta)x_2) = \theta(b_i - a_i^T x_!) + (1-\theta)(b_i - a_i^T x_2)$$

But we know that $b_i - a_i^T x_1 > 0$ and $b_i - a_i^T x_2 > 0$ and $0 \leq \theta \leq 1$, so

$$b_i - a_i^T(\theta x_1 + (1-\theta)x_2) > 0$$

and $\theta x_1 + (1-\theta)x_2 \in$ **domf** so **domf** is convex.
Then, it was asked to prove that $f(x)$ is a convex function.
First of all we know that $\mathbf{c}^T\mathbf{x}$ and $b_i - \mathbf{a}_i^T\mathbf{x}, i = 1, 2, ..., m$ are all affine and therefore convex and concave. Second, we know that $log(b_i - \mathbf{a}_i^T\mathbf{x})$ is concave in x because it is the composition of a known concave function. So the minus part of the logarithm is a convex function. Therefore, we proved that $f(x)$ is convex, as it is the sum of two convex quantities.
We continue to minimize f using cvx. Below, we see the constraints we inserted in order to get a solution for the problem. The constraints were derived from the notes that were indicated. The problem was run for $n = 2$.

Code:

```
%% C
close all;
clear all;
n = 2;
m = 20;
A = randn(m,n);
c = randn(n,1);
b = abs(randn(m,1));

% we define f as
f = @(x) c' * x - sum(log(b - A * x));

% a
cvx_begin
variable x(n)
minimize((c')*x - sum(log(b-(A)*x)))
subject to
b - A*x >= 0;
cvx_end
```

```
Successive approximation method to be employed.
   For improved efficiency, SDPT3 is solving the dual problem.
   SDPT3 will be called several times to refine the solution.
   Original size: 80 variables, 22 equality constraints
   20 exponentials add 160 variables, 100 equality constraints
------------------------------------------------------------------
 Cones  |             Errors              |
Mov/Act | Centering  Exp cone   Poly cone | Status
--------+---------------------------------+---------
 20/ 20 | 4.493e+00  1.142e+00  0.000e+00 | Solved
 20/ 20 | 9.417e-01  5.788e-02  0.000e+00 | Solved
 20/ 20 | 7.563e-02  3.674e-04  0.000e+00 | Solved
 19/ 20 | 9.706e-03  6.066e-06  0.000e+00 | Solved
 10/ 19 | 1.222e-03  9.607e-08  0.000e+00 | Solved
  0/ 10 | 1.544e-04  1.438e-09  0.000e+00 | Solved
------------------------------------------------------------------
Status: Solved
Optimal value (cvx_optval): +9.86668
```

Figure 11: CVX solution for our problem

Then, considering $n = 2$ we plotted f and its levels sets. First, we created a function just to check if a point x belongs to the **domf** and if the point isn't in the domain we give f in that point the value $f(x) = 1000$.
Code:

```
% function to check if a point belongs to domf
function domain_check = dom_f(A, b, c, x)

domain_check = true;

for i = 1:size(b,1)
    if(b(i) - A(i,:)*x <= 0)
        domain_check = false;
        break;
    end
end

end

% b
if (n==2)
    x1 = -1:0.01:1;
    x2 = -1:0.01:1;
    domf = all((b-A*x) > 0);
    %x_v = [x1; x2];
    fc = zeros(size(x1,2),size(x2,2));
    for i=1:size(x1,2)
        for j = 1:size(x2,2)
            if (b - A * [x1(i) ;x2(j)] > 0)
                fc(i, j) = c' * [x1(i) ; x2(j)]
                - sum(log(b - A * [x1(i);x2(j)]));
            else
                fc(i,j) = 1e3;
            end
        end
    end

    figure;
    contour(x1, x2, fc');
    xlabel('x1');
    ylabel('x2');
    title('Level Sets of f(x)');
    grid on;
end
```

What we observe from the plots, is that the level sets of the function approximate the form of a polyhedron.

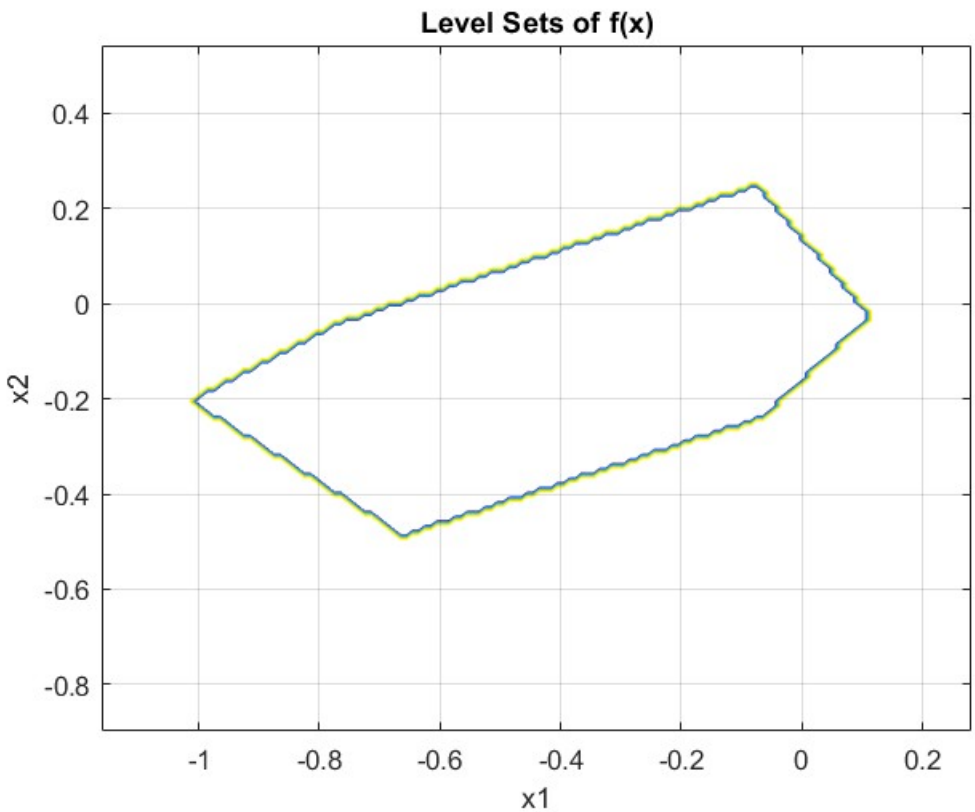

Figure 12: Level Sets of f(x)

Moving on to the next question, we implemented the gradient backtracking algorithm, assuming first that $\mathbf{x} = \mathbf{0}$ is a feasible point. Then, we started from the point $\mathbf{x} = \mathbf{0}$ and we proceeded to the backtracking line search following the method that was indicated. The code is presented below.
Code:

```
% c
gradf = @(x) c + sum(A./(b - (A)*x),1)';
x0 = zeros(n,1);
t = 1;
a = 0.2;
beta = 0.5;
e = 0.001;
[x,fun_val,f_val,all_x_kb,k] =
```

```
gradient_method_backtracking(f,gradf,x0,t,a,beta,e,A,b,n,c);

% modified backtracking function
function [x,fun_val,f_val,all_x_kb,iter]=
gradient_method_backtracking(f,g,x0,s,alpha,beta,epsilon,A,b,n,c)
x=x0;
grad =  g(x);
fun_val= f(x);
iter=0;
all_x_kb = zeros(n,iter);
f_val = zeros(1, iter);
ft = 1e3;
domain_check = true;

while(norm(grad) > epsilon)

    iter=iter+1;
    t = s;

    domain_check = dom_f(A,b,c,x);
    % that was proved unecessary
    while (domain_check == false)
        fprintf('anto');
        t = beta * t;
        x = x -t*grad;
        domain_check = dom_f(A,b,c,x);
    end

    % domain_check = dom_f(A,b,c,x-t*grad);

    while (fun_val - ft < alpha*t*norm(grad)^2)
        if (dom_f(A,b,c,x-t*grad) == false)

            ft = 1e3;
        else
            ft = f(x - t*grad);
        end

        t = beta*t;
```

```
    end

    x=x-t*grad;
    all_x_kb(:,iter) = x;
    f_val(iter) = f(x);
    fun_val=f(x);
    grad= g(x);
end
%fprintf('Grad = %f %f \n', grad);
fprintf('Optimal value f(x) = %f \n', fun_val);
fprintf('Iterations needed for backtracking = %d \n',iter);
end
```

Then, we moved on to implement the Newton algorithm using the steps that were presented in theory. Also, a function for the computation of the hessian was constructed.
Code:

```
% function for the computing of the hessian
function hessian = h(A, b, c, x)

    hf = zeros(size(c,1),size(c,1));

    for i = 1:size(b,1)
        hf = hf + (1/(b(i)-A(i,:)*x)^2)*(A(i,:)'*A(i,:));
    end

    hessian = hf;

end
```

```
% Newton's method
function [x,fun_val,f_val,all_x_kb,iter]=
gradient_method_newton(f,g,hf,x0,s,alpha,beta,epsilon,A,b,n,c)
x=x0;
hessian = h(A,b,c,x);
grad = g(x);
fun_val= f(x);
iter=0;
dxn = -inv(hessian)*grad;
lambda = grad'*inv(hessian)*grad;
ft = 1e3;
domain_check = true;

all_x_kb = zeros(n,iter);
f_val = zeros(1, iter);
t=s;

while (lambda > 2*epsilon)
    iter=iter+1;
    %dxn = -(inv(hessian))*grad;
    domain_check = dom_f(A,b,c,x);
    while (domain_check == false)
        t = beta * t;
        x = x + t*dxn;
        domain_check = dom_f(A,b,c,x+ t*dxn);
    end
    while (fun_val - ft >  norm(dxn)*t*grad' *grad)
        if (dom_f(A,b,c,x +t*dxn) == false)
            ft = 1e3;
        else
            ft = f(x + t*dxn);

        end
        t = beta*t;
    end

    x=x+t*dxn;
    all_x_kb(:,iter) = x;
    f_val(iter) = f(x);
    fun_val=f(x);
    grad=g(x);
```

```
    hessian = h(A,b,c,x);
    lambda=  grad'*inv(hessian)*grad;
    dxn = -inv(hessian)*grad;
end

fprintf('Optimal value f(x) = %f \n', fun_val);
fprintf('Iterations needed for Newton = %d \n',iter);
end
```

Below, are presented the results of each algorithm and the cvx minimization. Those results happened for $(n, m) = (2, 20)$.

```
 Cones  |             Errors              |
Mov/Act | Centering  Exp cone   Poly cone | Status
--------+---------------------------------+---------
 20/ 20 | 3.775e+00  8.226e-01  4.648e-10 | Solved
 20/ 20 | 8.606e-01  4.874e-02  0.000e+00 | Solved
 20/ 20 | 7.656e-02  3.748e-04  6.299e-10 | Solved
 19/ 20 | 1.034e-02  6.857e-06  4.624e-10 | Solved
  9/ 15 | 1.287e-03  1.068e-07  4.778e-10 | Solved
  0/  7 | 1.624e-04  2.095e-09  4.728e-10 | Solved
------------------------------------------------------------------
Status: Solved
Optimal value (cvx_optval): +8.58375

Optimal value f(x) = 8.583750
Iterations needed for backtracking = 24
Optimal value f(x) = 8.583888
Iterations needed for Newton = 6
```

Figure 13: Results for (n,m) = (2,20)

Then, we plotted the quantities $f_{gradient}(x_k) - p_*$ and $f_{Newton}(x_k) - p_*$ as a function of step k.

Code:

```
% e
k_axis = 1:k-1;
itern_axis = 1:itern-1;
quant_back_line = (f_val(1:k-1) - f(x_cvx_optimal));
quant_newton_line = (f_valn(1:itern-1) - f(x_cvx_optimal));

figure;
semilogy(k_axis, quant_back_line);
grid on;
title('$(f(\mathbf{x}_k) - p_*)$','Interpreter','latex');
xlabel('Iteration');
ylabel('(f_k - f_{cvx})');

figure;
semilogy(itern_axis, quant_newton_line);
grid on;
title('$(f(\mathbf{x}_k) - p_*)$','Interpreter','latex');
xlabel('Iteration');
ylabel('(f_k - f_{cvx})');
```

The plots are for $(n, m) = (2, 20)$:

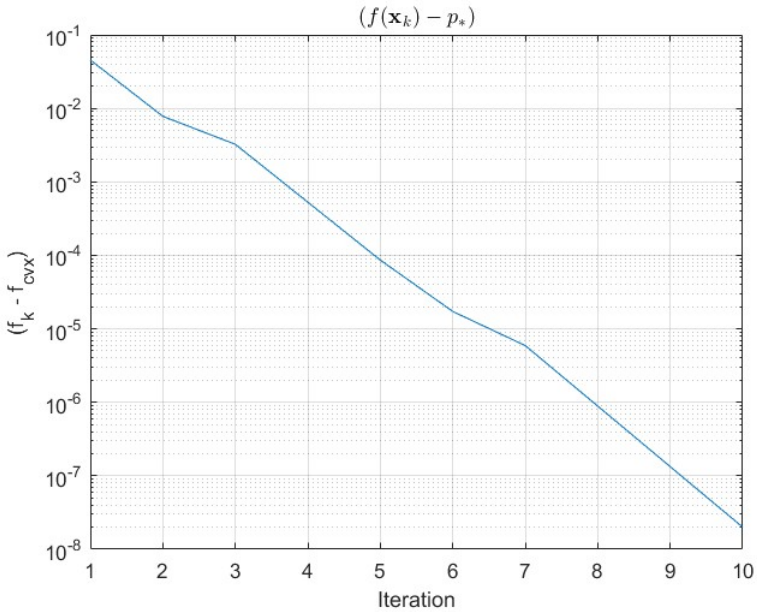

Figure 14: $f_{gradient}(x_k) - p_*$

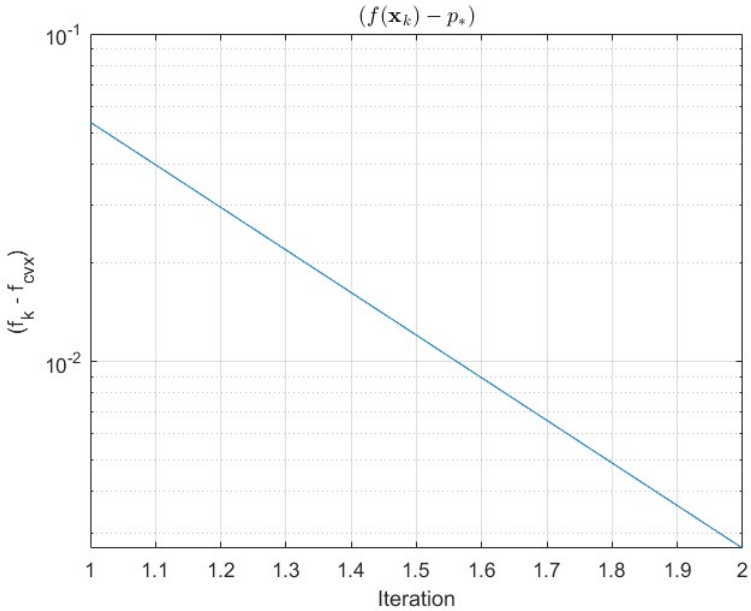

Figure 15: $f_{Newton}(x_k) - p_*$

Later, we run the algorithms for $(n, m) = (50, 200)$ and we got the following results:

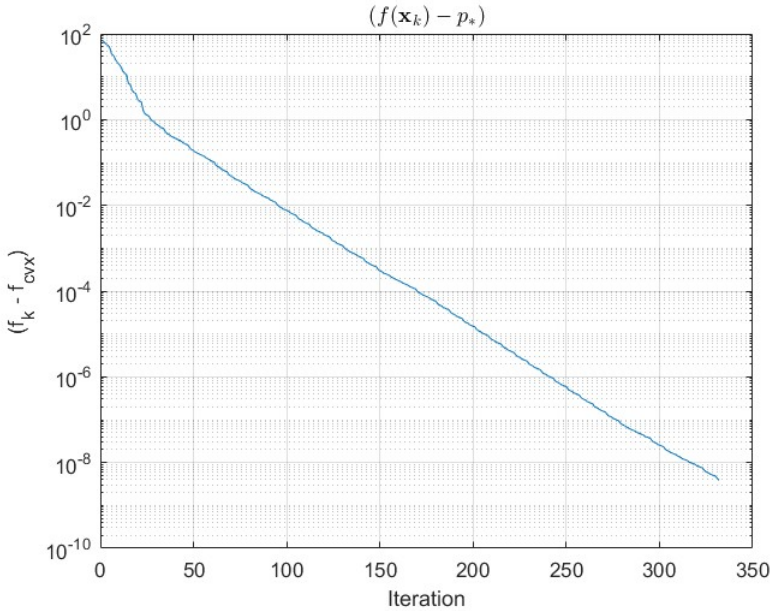

Figure 16: $f_{gradient}(x_k) - p_*$

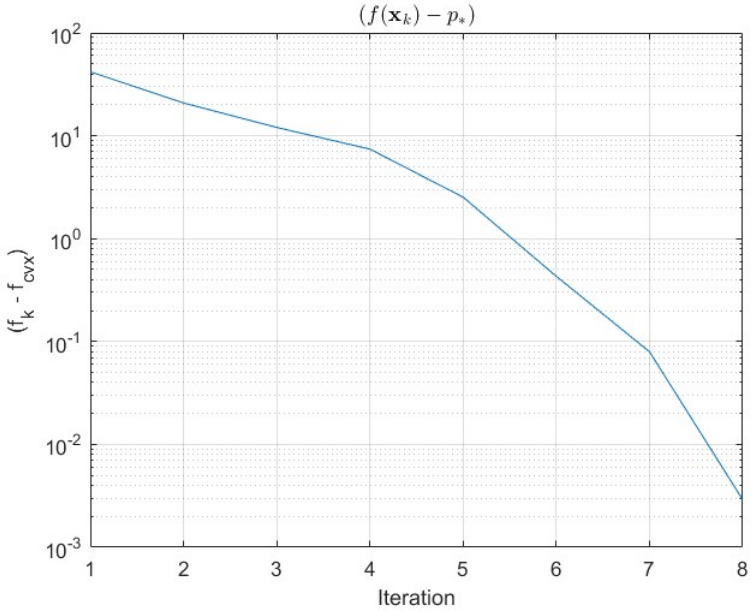

Figure 17: $f_{Newton}(x_k) - p_*$

We see that for small values of the pair (n, m) the lines are almost straight with the Newton one being completely straight. As the values of the pair are increasing,presented below, we observe edges to the lines. This happens because the iterations are increasing as well, which means that the inaccuracy is increasing. Also, another observation is that for bigger values of (n, m) the iterations of the backtracking algorithm are increasing in a much bigger degree from the iterations in the Newton algorithm.

```
 Cones  |              Errors              |
Mov/Act | Centering  Exp cone   Poly cone | Status
--------+---------------------------------+---------
200/200 | 4.430e+00  1.112e+00  0.000e+00 | Solved
200/200 | 8.953e-01  5.226e-02  0.000e+00 | Solved
196/198 | 8.505e-02  4.648e-04  0.000e+00 | Solved
178/197 | 1.085e-02  7.586e-06  0.000e+00 | Solved
 70/166 | 1.336e-03  1.146e-07  0.000e+00 | Solved
  0/ 56 | 1.662e-04  1.469e-09  0.000e+00 | Solved
------------------------------------------------------------------
Status: Solved
Optimal value (cvx_optval): +65.5631

Optimal value f(x) = 65.563089
Iterations needed for backtracking = 333
Optimal value f(x) = 65.563093
Iterations needed for Newton = 9
```

Figure 18: Results for $(n, m) = (50, 200)$

As we see the beginning of the backtracking line is much more steeper than that in the Newton line. Here we observe that the quantity produced by the backtracking algorithm is resembling to a straight line. The Newton algorithm doesn't produce such a steep slope, but it needs just 8 or 9 iterations to solve the problem. However, the time complexity of the Newton algorithm is bigger due to the inversion of the Hessian.